CUDA Scan (Prefix Sum) - Parallel Prefix Algorithms

학습 목표

- Scan (Prefix Sum) 개념 이해
- Exclusive vs Inclusive scan 차이
- Naive vs Work-Efficient 알고리즘
- 실전 응용 사례 파악
- GPU 최적화 기법

예상 학습 시간: 3-4시간
난이도: ⭐ ⭐ ⭐ (중급-고급)
선수 과목: 00-03 CUDA 기초, Thread Synchronization

---

1. Scan이란?

1.1 정의

Scan (Prefix Sum): 배열의 각 원소에 대해, 그 이전 모든 원소들의 누적 합을 계산

```
입력:  [3, 1, 7, 0, 4, 1, 6, 3]

Inclusive Scan:
출력:  [3, 4, 11, 11, 15, 16, 22, 25]
       ↑  ↑   ↑   ↑   ↑   ↑   ↑   ↑
       3  3+1 +7  +0  +4  +1  +6  +3

Exclusive Scan:
출력:  [0, 3, 4, 11, 11, 15, 16, 22]
       ↑  ↑  ↑   ↑   ↑   ↑   ↑   ↑
```

```
0  3  3+1 +7  +0  +4  +1  +6
(identity)
```

Inclusive: 자신 포함
Exclusive: 자신 제외 (왼쪽까지만)

1.2 일반화

덧셈 외에 다른 연산자도 가능:

```
// 곱셈 scan
input:  [2, 3, 1, 4]
output: [2, 6, 6, 24]  // 2, 2*3, 2*3*1, 2*3*1*4

// 최대값 scan
input:  [3, 1, 7, 4]
output: [3, 3, 7, 7]   // max(3), max(3,1), max(3,1,7), ...

// 논리 OR scan
input:  [0, 1, 0, 0]
output: [0, 1, 1, 1]   // 0, 0|1, 0|1|0, ...
```

요구사항: 연산자가 **associative** (결합법칙)

- ✅ 덧셈: (a+b)+c = a+(b+c)
- ✅ 곱셈, max, min, OR, AND
- ❌ 나눗셈: (a/b)/c ≠ a/(b/c)

---

2. Scan의 응용

2.1 Stream Compaction

```
// 문제: 0이 아닌 원소만 남기기
input:  [3, 0, 0, 1, 0, 4, 0, 2]
desired: [3, 1, 4, 2]

// 해법: Scan 사용
flags:  [1, 0, 0, 1, 0, 1, 0, 1]  // 0이 아니면 1
scan:   [0, 1, 1, 1, 2, 2, 3, 3]  // Exclusive scan
```

```
        ↑           ↑     ↑     ↑
output[0] = input[0] = 3
output[2] = input[3] = 1
output[3] = input[5] = 4
output[4] = input[7] = 2
```

응용: Sparse matrix, ray tracing, collision detection

2.2 Radix Sort

```
// Radix sort는 scan을 반복 사용
// 각 비트 위치마다 stable partition

// 예: 0비트로 정렬
input:  [5, 2, 7, 4, 1, 6]  // Binary: 101, 010, 111, 100, 001, 110
bit 0:  [1, 0, 1, 0, 1, 0]  // Odd = 1, Even = 0

// 0인 원소들의 위치 (exclusive scan)
scan_0: [0, 0, 1, 1, 2, 2]  // scan of (1-bit)
// 1인 원소들의 위치
scan_1: [0, 1, 1, 2, 2, 3]  // scan of bit

// 결과: [2, 4, 6, 5, 7, 1]  // Even first, then odd
```

2.3 Work Distribution

```
// 문제: 각 thread가 가변 개수의 작업 수행
work_counts: [5, 2, 8, 3, 0, 4, 1, 6]  // Thread i가 할 작업 개수

// Scan으로 시작 위치 계산
start_indices: [0, 5, 7, 15, 18, 18, 22, 23]
               ↑  ↑  ↑   ↑   ↑   ↑   ↑   ↑
// Thread 0: work[0-4]   (5개)
// Thread 1: work[5-6]   (2개)
// Thread 2: work[7-14]  (8개)
// ...
```

2.4 기타 응용

- **Lexical analysis**: Token boundary detection
- **String matching**: Pattern counting

- **Graph algorithms**: BFS level assignment
- **Image processing**: Integral image (SAT)
- **Dynamic programming**: Parallel DP

---

3. Naive Scan (Sequential)

3.1 CPU 구현

```
// Inclusive scan
void scan_inclusive_cpu(int* input, int* output, int N) {
    output[0] = input[0];
    for (int i = 1; i < N; i++) {
        output[i] = output[i-1] + input[i];
    }
}

// Exclusive scan
void scan_exclusive_cpu(int* input, int* output, int N) {
    output[0] = 0;  // Identity element
    for (int i = 1; i < N; i++) {
        output[i] = output[i-1] + input[i-1];
    }
}

// 시간 복잡도: O(N)
// 병렬화 불가능! (각 iteration이 이전 결과에 의존)
```

3.2 문제점

```
output[i] = output[i-1] + input[i]
            └ 의존성! i번째를 계산하려면 (i-1)번째 필요
```

해결책: Tree-based 알고리즘으로 병렬화!

---

4. Hillis-Steele Scan (Step-Efficient)

4.1 알고리즘 개요

아이디어: 각 step마다 거리를 2배씩 늘려가며 더하기

```
입력: [3, 1, 7, 0, 4, 1, 6, 3]

Step 0 (거리=1):
[3, 3+1, 1+7, 7+0, 0+4, 4+1, 1+6, 6+3]
= [3, 4, 8, 7, 4, 5, 7, 9]

Step 1 (거리=2):
[3, 4, 3+8, 4+7, 8+4, 7+5, 4+7, 5+9]
= [3, 4, 11, 11, 12, 12, 11, 14]

Step 2 (거리=4):
[3, 4, 11, 11, 3+12, 4+12, 11+11, 11+14]
= [3, 4, 11, 11, 15, 16, 22, 25]  ✓ 완료!
```

4.2 CUDA 구현 (Naive)

```
__global__ void scan_hillis_steele(float* input, float* output, int N) {
    extern __shared__ float temp[];
    int tid = threadIdx.x;

    // Load input
    temp[tid] = (tid < N) ? input[tid] : 0;
    __syncthreads();

    // Up-sweep phase
    for (int stride = 1; stride < blockDim.x; stride *= 2) {
        float val = 0;
        if (tid >= stride) {
            val = temp[tid - stride];
        }
        __syncthreads();

        temp[tid] += val;
        __syncthreads();
    }

    // Write output
    if (tid < N) {
        output[tid] = temp[tid];
```

```
    }
}
```

4.3 특징

장점:

- ✅ Step 개수가 적음: O(log N) steps
- ✅ 구현이 간단

단점:

- ❌ Work inefficient: O(N log N) 연산
  - 8개 원소: 3 steps × 8 threads = 24 ops (vs CPU 7 ops)
- ❌ 메모리 대역폭 낭비

---

5. Blelloch Scan (Work-Efficient)

5.1 알고리즘 개요

아이디어: Up-sweep (reduction) + Down-sweep (distribution)

Phase 1: Up-Sweep (Reduce Phase)

```
입력: [3, 1, 7, 0, 4, 1, 6, 3]

Level 0 (stride=1):
  [_, 3+1, _, 7+0, _, 4+1, _, 6+3]
= [3, 4, 7, 7, 4, 5, 6, 9]

Level 1 (stride=2):
  [_, _, _, 4+7, _, _, _, 5+9]
= [3, 4, 7, 11, 4, 5, 6, 14]

Level 2 (stride=4):
  [_, _, _, _, _, _, _, 11+14]
= [3, 4, 7, 11, 4, 5, 6, 25]  ← 마지막 원소 = 전체 합
```

Phase 2: Down-Sweep (Distribute Phase)

```
시작: [3, 4, 7, 11, 4, 5, 6, 0]  ← 마지막을 0으로 교체

Level 2 (stride=4):
  [_, _, _, 0, _, _, _, 11]
= [3, 4, 7, 0, 4, 5, 6, 11]

Level 1 (stride=2):
  [_, _, 0, 7, _, _, 11, 14]
= [3, 4, 0, 7, 4, 5, 11, 14]

Level 0 (stride=1):
  [0, 3, 4, 11, 11, 15, 16, 22]  ✓ Exclusive scan 완료!
```

5.2 CUDA 구현

```
__global__ void scan_blelloch(float* data, int N) {
    extern __shared__ float temp[];
    int tid = threadIdx.x;

    // Load data
    int offset = 1;
    temp[2*tid]     = (2*tid < N)   ? data[2*tid]   : 0;
    temp[2*tid + 1] = (2*tid+1 < N) ? data[2*tid+1] : 0;

    // Up-sweep (reduction) phase
    for (int d = N >> 1; d > 0; d >>= 1) {
        __syncthreads();
        if (tid < d) {
            int ai = offset * (2*tid + 1) - 1;
            int bi = offset * (2*tid + 2) - 1;
            temp[bi] += temp[ai];
        }
        offset *= 2;
    }

    // Clear last element (exclusive scan)
    if (tid == 0) {
        temp[N - 1] = 0;
    }

    // Down-sweep phase
    for (int d = 1; d < N; d *= 2) {
```

```
        offset >>= 1;
        __syncthreads();
        if (tid < d) {
            int ai = offset * (2*tid + 1) - 1;
            int bi = offset * (2*tid + 2) - 1;

            float t = temp[ai];
            temp[ai] = temp[bi];
            temp[bi] += t;
        }
    }
    __syncthreads();

    // Write back
    if (2*tid < N)   data[2*tid]   = temp[2*tid];
    if (2*tid+1 < N) data[2*tid+1] = temp[2*tid+1];
}
```

5.3 특징

장점:

- ✅ Work efficient: O(N) 연산 (CPU와 동일!)
- ✅ Step efficient: O(log N) steps

단점:

- ❌ 구현이 복잡
- ❌ Bank conflicts 발생 가능
- ❌ 작은 N에서는 오버헤드

5.4 복잡도 비교

알고리즘	Work (연산)	Steps	병렬성
Sequential	O(N)	O(N)	1
Hillis-Steele	O(N log N)	O(log N)	O(N)
Blelloch	O(N)	O(log N)	O(N/log N)

---

6. Bank Conflict 회피

6.1 문제

```
// Blelloch에서 stride가 커질수록 bank conflict
// stride = 16일 때:
temp[15] += temp[31];  // Bank 15, Bank 31
temp[47] += temp[63];  // Bank 15, Bank 31 → Conflict!
```

6.2 해결: Padding

```
#define NUM_BANKS 32
#define CONFLICT_FREE_OFFSET(n) ((n) / NUM_BANKS)

// Allocation
int size = n + CONFLICT_FREE_OFFSET(n);
__shared__ float temp[size];

// Access
int ai = tid;
int bi = tid + (n / 2);
int bankOffsetA = CONFLICT_FREE_OFFSET(ai);
int bankOffsetB = CONFLICT_FREE_OFFSET(bi);

temp[ai + bankOffsetA] = data[ai];
temp[bi + bankOffsetB] = data[bi];
```

---

7. Multi-Block Scan

7.1 문제

- Shared memory 크기 제한 (48-128 KB)
- Block당 최대 1024-2048 원소만 scan 가능
- N > block size인 경우?

7.2 해결: 3-Phase Algorithm

```
입력: N = 8192 원소 (4 blocks × 2048)

Phase 1: Block-level scan
Block 0: scan [0:2047]    → sum_0 = 1000
Block 1: scan [2048:4095] → sum_1 = 1500
Block 2: scan [4096:6143] → sum_2 = 800
Block 3: scan [6144:8191] → sum_3 = 1200

Phase 2: Scan block sums
sums: [1000, 1500, 800, 1200]
scan: [0, 1000, 2500, 3300]  ← 각 block의 offset

Phase 3: Add offsets
Block 0: += 0
Block 1: += 1000
Block 2: += 2500
Block 3: += 3300
```

7.3 구현

```
// Phase 1: Block-level scans
scan_kernel<<<num_blocks, block_size>>>(d_input, d_output, d_block_sums, N);

// Phase 2: Scan of block sums (single block)
scan_kernel<<<1, num_blocks>>>(d_block_sums, d_block_sums, NULL, num_blocks);

// Phase 3: Add offsets
add_kernel<<<num_blocks, block_size>>>(d_output, d_block_sums, N);
```

---

8. 최적화 기법

8.1 벡터화 로드/스토어

```
// 한 번에 4개 원소 로드
float4 val = reinterpret_cast<float4*>(input)[tid];
temp[4*tid]   = val.x;
temp[4*tid+1] = val.y;
temp[4*tid+2] = val.z;
temp[4*tid+3] = val.w;
```

8.2 Loop Unrolling

```
// Up-sweep unrolled
#pragma unroll
for (int d = N >> 1; d > 32; d >>= 1) {
    // ...
}
// Last 5 iterations unrolled manually
if (tid < 16) temp[bi] += temp[ai];
if (tid < 8)  temp[bi] += temp[ai];
// ...
```

8.3 Warp-Level Scan

```
__device__ int warp_scan_inclusive(int val) {
    #pragma unroll
    for (int offset = 1; offset < 32; offset <<= 1) {
        int n = __shfl_up_sync(0xffffffff, val, offset);
        if (threadIdx.x % 32 >= offset) val += n;
    }
    return val;
}
```

---

9. CUB Library

9.1 Production-Ready Scan

```
#include <cub/cub.cuh>

// Device-level scan (any size)
void* d_temp_storage = NULL;
size_t temp_storage_bytes = 0;

// Query temp storage size
cub::DeviceScan::ExclusiveSum(d_temp_storage, temp_storage_bytes,
                              d_in, d_out, N);

// Allocate temp storage
```

```
cudaMalloc(&d_temp_storage, temp_storage_bytes);

// Run scan
cub::DeviceScan::ExclusiveSum(d_temp_storage, temp_storage_bytes,
                              d_in, d_out, N);

// Performance: Near-optimal (99% of theoretical peak)
```

9.2 Block-Level Scan (in Kernel)

```
__global__ void my_kernel(int* data, int N) {
    typedef cub::BlockScan<int, 256> BlockScan;
    __shared__ typename BlockScan::TempStorage temp_storage;

    int thread_data = data[threadIdx.x];
    BlockScan(temp_storage).ExclusiveSum(thread_data, thread_data);
    data[threadIdx.x] = thread_data;
}
```

---

10. 실습 예제

다음 파일 참고:

- `examples/cuda/06_scan.cu` - Blelloch scan 구현
- `examples/cuda/07_scan_applications.cu` - Stream compaction 등

요약 체크리스트

- [ ] Inclusive vs Exclusive scan 차이 이해
- [ ] Scan의 5가지 이상 응용 사례 파악
- [ ] Hillis-Steele 알고리즘 이해
- [ ] Blelloch 알고리즘 완벽 이해 (Up-sweep + Down-sweep)
- [ ] Work efficiency vs Step efficiency 차이 파악
- [ ] Bank conflict 회피 방법 숙지
- [ ] Multi-block scan 3-phase 알고리즘 이해

- [ ] CUB library 사용법 숙지

핵심 개념:

- Scan = Fundamental parallel primitive
- Blelloch = Work-efficient O(N)
- Applications: Compaction, sorting, work distribution
- CUB = Production-ready implementation